

```
aLast login: Thu Apr 30 12:58:31 on console
thanhithpuppala@Thanhiths-MacBook-Air ~ % mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.45 MySQL Community Server - GPL
```

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> USE db_world
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

Database changed

```
mysql> CREATE TABLE offices (
-> officeCode VARCHAR(10) PRIMARY KEY,
-> city VARCHAR(50),
-> phone VARCHAR(50),
-> addressLine1 VARCHAR(100),
-> addressLine2 VARCHAR(100),
-> state VARCHAR(50),
-> country VARCHAR(50),
-> postalCode VARCHAR(15),
->
-> territory VARCHAR(50)
-> );
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> CREATE TABLE employees (
-> employeeNumber INT PRIMARY KEY,
-> lastName VARCHAR(50),
-> firstName VARCHAR(50),
-> extension VARCHAR(10),
-> email VARCHAR(100),
-> officeCode VARCHAR(10),
-> reportsTo INT,
-> jobTitle VARCHAR(50),
-> FOREIGN KEY (officeCode) REFERENCES offices(officeCode),
-> FOREIGN KEY (reportsTo) REFERENCES employees(employeeNumber)
-> );
```

ERROR 1050 (42S01): Table 'employees' already exists

```
mysql> DROP TABLE employees;
```

Query OK, 0 rows affected (0.01 sec)

```
mysql> CREATE TABLE employees (
-> employeeNumber INT PRIMARY KEY,
-> lastName VARCHAR(50),
-> firstName VARCHAR(50),
-> extension VARCHAR(10),
-> email VARCHAR(100),
-> officeCode VARCHAR(10),
-> reportsTo INT,
-> jobTitle VARCHAR(50),
-> FOREIGN KEY (officeCode) REFERENCES offices(officeCode),
-> FOREIGN KEY (reportsTo) REFERENCES employees(employeeNumber)
-> );
```

Query OK, 0 rows affected (0.02 sec)

```
mysql> CREATE TABLE customers (  
-> customerNumber INT PRIMARY KEY,  
-> customerName VARCHAR(100),  
-> contactLastName VARCHAR(50),  
-> contactFirstName VARCHAR(50),  
-> phone VARCHAR(50),  
-> addressLine1 VARCHAR(100),  
-> addressLine2 VARCHAR(100),  
-> city VARCHAR(50),  
-> state VARCHAR(50),  
-> postalCode VARCHAR(15),  
-> country VARCHAR(50),  
-> salesRepEmployeeNumber INT,  
-> creditLimit DECIMAL(10,2),  
-> FOREIGN KEY (salesRepEmployeeNumber) REFERENCES  
employees(employeeNumber)  
-> );  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> CREATE TABLE productlines (  
-> productLine VARCHAR(50) PRIMARY KEY,  
-> textDescription TEXT,  
-> htmlDescription TEXT,  
-> image TEXT  
-> );  
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> -- PRODUCTS  
mysql> CREATE TABLE products (  
-> productCode VARCHAR(15) PRI\c  
mysql> CREATE TABLE products (  
-> productCode VARCHAR(15) PRIMARY KEY,  
->  
-> productName VARCHAR(100),  
-> productLine VARCHAR(50),  
-> productScale VARCHAR(10),  
-> productVendor VARCHAR(50),  
-> productDescription TEXT,  
-> quantityInStock INT,  
-> buyPrice DECIMAL(10,2),  
-> MSRP DECIMAL(10,2),  
-> FOREIGN KEY (productLine) REFERENCES productlines(productLine)  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> CREATE TABLE orders (  
-> orderNumber INT PRIMARY KEY,  
-> orderDate DATE,  
-> requiredDate DATE,  
-> shippedDate DATE,  
-> status VARCHAR(20),  
-> comments TEXT,  
-> customerNumber INT,  
-> FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> CREATE TABLE orderdetails (  
-> orderNumber INT,  
-> productCode VARCHAR(15),  
-> quantityOrdered INT,  
-> priceEach DECIMAL(10,2),  
-> orderLineNumber INT,
```

```

    -> PRIMARY KEY (orderNumber, productCode),
    -> FOREIGN KEY (orderNumber) REFERENCES orders(orderNumber),
    -> FOREIGN KEY (productCode) REFERENCES products(productCode)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE payments (
    -> customerNumber INT,
    -> checkNumber VARCHAR(50),
    -> paymentDate DATE,
    -> amount DECIMAL(10,2),
    -> PRIMARY KEY (customerNumber, checkNumber),
    -> FOREIGN KEY (customerNumber) REFERENCES customers(customerNumber)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO offices VALUES
    -> ('1', 'New York', '1234567890', '5th Ave', NULL, 'NY', 'USA', '10001',
'NA'),
    -> ('2', 'London', '9876543210', 'Baker St', NULL, NULL, 'UK', 'NW1',
'EMEA');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO employees VALUES
    -> (1001, '&#39;Smith&#39;;, &#39;John&#39;;, &#39;x101&#39;;,
&#39;john@company.com&#39;;, &#39;1&#39;;, NULL, &#39;Manager&#39;);,
    -> (1002, '&#39;Doe&#39;;, &#39;Jane&#39;;, &#39;x102&#39;;,
&#39;jane@company.com&#39;;, &#39;1&#39;;, 1001, &#39;Sales Rep&#39;);,
    -> (1003, '&#39;Brown&#39;;, &#39;Mike&#39;;, &#39;x103&#39;;,
&#39;mike@company.com&#39;;, &#39;2&#39;;, 1001, &#39;Sales Rep&#39;);,
    ->
    -> INSERT INTO employees VALUES
    -> (1001, 'Smith', 'John', 'x101', 'john@company.com', '1', NULL,
'Manager'),
    -> (1002, 'Doe', 'Jane', 'x102', 'jane@company.com', '1', 1001, 'Sales
Rep'),
    -> (1003, 'Brown', 'Mike', 'x103', 'mike@company.com', '2', 1001, 'Sales
Rep');\c
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near '&
(1002, &
(1003, &

INSERT INTO employees VALUES
(1001, 'Smith', 'John', 'x101' at line 2
mysql> INSERT INTO employees VALUES
    -> (1001, 'Smith', 'John', 'x101', 'john@company.com', '1', NULL,
'Manager'),
    -> (1002, 'Doe', 'Jane', 'x102', 'jane@company.com', '1', 1001, 'Sales
Rep'),
    -> (1003, 'Brown', 'Mike', 'x103', 'mike@company.com', '2', 1001, 'Sales
Rep');
Query OK, 3 rows affected (0.00 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO customers VALUES
    -> (2001, 'ABC Corp', 'Lee', 'Chris', '1111111111', 'Street 1', NULL,
'NYC', 'NY', '10001', 'USA', 1002, 50000),
    -> (2002, 'XYZ Ltd', 'Kim', 'Pat', '2222222222', 'Street 2', NULL,
'London', NULL, 'NW1', 'UK', 1003, 70000),
    -> (2003, 'Global Inc', 'Singh', 'Raj', '3333333333', 'Street 3', NULL,
'Delhi', NULL, '110001', 'India', 1002, 60000);

```

Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> Product Lines

```
-> INSERT INTO productlines VALUES
-> ('Cars', 'Car models', NULL, NULL),
-> ('Bikes', 'Bike models', NULL, NULL);
-> \c
```

```
mysql> INSERT INTO productlines VALUES
-> ('Cars', 'Car models', NULL, NULL),
-> ('Bikes', 'Bike models', NULL, NULL);
```

Query OK, 2 rows affected (0.00 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> INSERT INTO products VALUES

```
-> ('P101', 'Toyota Camry', 'Cars', '1:18', 'Toyota', 'Sedan model', 100,
20.00, 30.00),
-> ('P102', 'Honda Civic', 'Cars', '1:18', 'Honda', 'Compact model', 150,
18.00, 28.00),
-> ('P201', 'Yamaha Bike', 'Bikes', '1:12', 'Yamaha', 'Sport bike', 200,
10.00, 15.00);
```

Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO orders VALUES

```
-> (3001, '2024-01-01', '2024-01-05', '2024-01-03', 'Shipped', NULL,
2001),
-> (3002, '2024-02-01', '2024-02-05', '2024-02-04', 'Shipped', NULL,
2002),
-> (3003, '2024-03-01', '2024-03-06', NULL, 'Pending', NULL, 2003);
-> \c
```

mysql> INSERT INTO orders VALUES

```
-> (3001, '2024-01-01', '2024-01-05', '2024-01-03', 'Shipped', NULL,
2001),
-> (3002, '2024-02-01', '2024-02-05', '2024-02-04', 'Shipped', NULL,
2002),
-> (3003, '2024-03-01', '2024-03-06', NULL, 'Pending', NULL, 2003);
```

Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO orderdetails VALUES

```
-> (3001, 'P101', 10, 30.00, 1),
-> (3001, 'P201', 5, 15.00, 2),
-> (3002, 'P102', 7, 28.00, 1),
-> (3003, 'P101', 3, 30.00, 1);
```

Query OK, 4 rows affected (0.00 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> INSERT INTO payments VALUES

```
-> (2001, 'CHK001', '2024-01-10', 300.00),
-> (2002, 'CHK002', '2024-02-10', 500.00),
-> (2003, 'CHK003', '2024-03-10', 150.00);
```

Query OK, 3 rows affected (0.00 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM customers

-> ORDER BY customerName;

```
+-----+-----+-----+-----+-----+
| customerNumber | customerName | contactLastName | contactFirstName | phone | |
| addressLine1 | addressLine2 | city | state | postalCode | country |
| salesRepEmployeeNumber | creditLimit |
```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|          2001 | ABC Corp      | Lee          | Chris        |
1111111111 | Street 1      | NULL         | NYC          | 10001        | USA
|          1002 |              | 50000.00     |              |
|          2003 | Global Inc    | Singh        | Raj          |
3333333333 | Street 3      | NULL         | Delhi        | 110001       |
India      |              | 1002         | 60000.00     |
|          2002 | XYZ Ltd       | Kim          | Pat          |
2222222222 | Street 2      | NULL         | London       | NULL         | NW1          | UK
|          1003 |              | 70000.00     |              |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

```

```

mysql> SELECT * FROM products
      -> ORDER BY buyPrice DESC;

```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| productCode | productName   | productLine | productScale | productVendor |
productDescription | quantityInStock | buyPrice | MSRP |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| P101        | Toyota Camry  | Cars        | 1:18         | Toyota        |
Sedan model   |              | 100         | 20.00 | 30.00 |
| P102        | Honda Civic   | Cars        | 1:18         | Honda         |
Compact model |              | 150         | 18.00 | 28.00 |
| P201        | Yamaha Bike   | Bikes       | 1:12         | Yamaha        |
Sport bike    |              | 200         | 10.00 | 15.00 |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> --Show all employees ordered by lastName
      -> SELECT * FROM employees
      -> ORDER BY lastName;

```

```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near '--Show all employees ordered by lastName
SELECT * FROM employees
ORDER BY lastNa' at line 1

```

```

mysql> -- Show all employees ordered by lastName
mysql> SELECT * FROM employees
      -> ORDER BY lastName;

```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
| employeeNumber | lastName | firstName | extension | email          |
officeCode | reportsTo | jobTitle |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
|          1003 | Brown    | Mike      | x103        | mike@company.com | 2
|          1001 | Sales Rep |           |             |                   |
|          1002 | Doe      | Jane      | x102        | jane@company.com | 1
|          1001 | Sales Rep |           |             |                   |
|          1001 | Smith    | John      | x101        | john@company.com | 1
|          NULL | Manager  |           |             |                   |
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- List all offices sorted by country
mysql> SELECT * FROM offices

```

```

-> ORDER BY country;
+-----+-----+-----+-----+-----+-----+
| officeCode | city      | phone      | addressLine1 | addressLine2 | state |
country | postalCode | territory |
+-----+-----+-----+-----+-----+-----+
| 2          | London    | 9876543210 | Baker St      | NULL          | NULL |
UK      | NW1        | EMEA        |
| 1          | New York  | 1234567890 | 5th Ave       | NULL          | NY   |
USA     | 10001      | NA          |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

```

```

mysql> -- Get all orders sorted by orderDate descending
mysql> SELECT * FROM orders
-> ORDER BY orderDate DESC;
+-----+-----+-----+-----+-----+-----+
| orderNumber | orderDate | requiredDate | shippedDate | status | comments |
customerNumber |
+-----+-----+-----+-----+-----+-----+
|          3003 | 2024-03-01 | 2024-03-06   | NULL       | Pending | NULL      |
|          2003 |
|          3002 | 2024-02-01 | 2024-02-05   | 2024-02-04 | Shipped | NULL      |
|          2002 |
|          3001 | 2024-01-01 | 2024-01-05   | 2024-01-03 | Shipped | NULL      |
|          2001 |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- Customer names with their sales representative
mysql> SELECT c.customerName, e.firstName, e.lastName
-> FROM customers c
-> JOIN employees e
-> ON c.salesRepEmployeeNumber = e.employeeNumber;
+-----+-----+-----+
| customerName | firstName | lastName |
+-----+-----+-----+
| ABC Corp     | Jane     | Doe      |
| XYZ Ltd      | Mike     | Brown    |
| Global Inc   | Jane     | Doe      |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- List all products with their product line
mysql> SELECT productName, productLine
-> FROM products;
+-----+-----+
| productName | productLine |
+-----+-----+
| Toyota Camry | Cars        |
| Honda Civic  | Cars        |
| Yamaha Bike  | Bikes       |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- Display orders with customer names
mysql> SELECT o.orderNumber, c.customerName
-> FROM orders o
-> JOIN customers c

```

```

-> ON o.customerNumber = c.customerNumber;
+-----+-----+
| orderNumber | customerName |
+-----+-----+
|          3001 | ABC Corp      |
|          3002 | XYZ Ltd       |
|          3003 | Global Inc    |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- Show order details with product names
mysql> SELECT od.orderNumber, p.productName, od.quantityOrdered
-> FROM orderdetails od
-> JOIN products p
-> ON od.productCode = p.productCode;
+-----+-----+-----+
| orderNumber | productName | quantityOrdered |
+-----+-----+-----+
|          3001 | Toyota Camry |             10 |
|          3003 | Toyota Camry |              3 |
|          3002 | Honda Civic  |              7 |
|          3001 | Yamaha Bike  |              5 |
+-----+-----+-----+
4 rows in set (0.00 sec)

```

```

mysql> -- 11.SELECT od.orderNumber, p.productName, od.quantityOrdered
mysql> FROM orderdetails od
-> JOIN products p
-> ON od.productCode = p.productCode;\c
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near 'FROM orderdetails od
JOIN products p
ON od.productCode = p.productCode' at line 1
mysql> -- 11.List payments with customer names
mysql> SELECT p.checkNumber, c.customerName, p.amount
-> FROM payments p
-> JOIN customers c
-> ON p.customerNumber = c.customerNumber;
+-----+-----+-----+
| checkNumber | customerName | amount |
+-----+-----+-----+
| CHK001      | ABC Corp      | 300.00 |
| CHK002      | XYZ Ltd       | 500.00 |
| CHK003      | Global Inc    | 150.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- 12.Count total number of orders
mysql> SELECT COUNT(*) AS total_orders FROM orders;
+-----+
| total_orders |
+-----+
|             3 |
+-----+
1 row in set (0.01 sec)

```

```

mysql> -- 13. Find average buyPrice of products
mysql> SELECT AVG(buyPrice) AS avg_price FROM products;
+-----+
| avg_price |
+-----+
| 16.000000 |
+-----+

```

1 row in set (0.00 sec)

```
mysql> -- 14. Find maximum MSRP from products
mysql> SELECT MAX(MSRP) AS max_price FROM products;
```

max_price
30.00

1 row in set (0.00 sec)

```
mysql> -- 15. Find total number of employees
mysql> SELECT COUNT(*) AS total_employees FROM employees;
```

total_employees
3

1 row in set (0.00 sec)

```
mysql> -- 16. Count customers per country
mysql> SELECT country, COUNT(*) AS total_customers
-> FROM customers
-> GROUP BY country;
```

country	total_customers
USA	1
UK	1
India	1

3 rows in set (0.00 sec)

```
mysql> -- 17. Count orders per status
mysql> SELECT status, COUNT(*) AS total_orders
-> FROM orders
-> GROUP BY status;
```

status	total_orders
Shipped	2
Pending	1

2 rows in set (0.00 sec)

```
mysql> -- 18. Count products per product line
mysql> SELECT productLine, COUNT(*) AS total_products
-> FROM products
-> GROUP BY productLine;
```

productLine	total_products
Bikes	1
Cars	2

2 rows in set (0.00 sec)

```
mysql> -- 19. Average credit limit per country
mysql> SELECT country, AVG(creditLimit) AS avg_credit
-> FROM customers
-> GROUP BY country;
```

country	avg_credit
---------	------------


```

| USA      | 50000.000000 |
| UK       | 70000.000000 |
| India    | 60000.000000 |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- 20. Count employees per office
mysql> SELECT officeCode, COUNT(*) AS total_employees
      -> FROM employees
      -> GROUP BY officeCode;
+-----+-----+
| officeCode | total_employees |
+-----+-----+
| 1          |                2 |
| 2          |                1 |
+-----+-----+
2 rows in set (0.00 sec)

```

```

mysql> -- 21. Count number of orders per customer
mysql> SELECT customerNumber, COUNT(orderNumber) AS total_orders
      -> FROM orders
      -> GROUP BY customerNumber;
+-----+-----+
| customerNumber | total_orders |
+-----+-----+
|          2001 |            1 |
|          2002 |            1 |
|          2003 |            1 |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- 22. Find total payment amount per customer
mysql> SELECT customerNumber, SUM(amount) AS total_payment
      -> FROM payments
      -> GROUP BY customerNumber;
+-----+-----+
| customerNumber | total_payment |
+-----+-----+
|          2001 |        300.00 |
|          2002 |        500.00 |
|          2003 |        150.00 |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- 23. List total quantity ordered per product
mysql> SELECT productCode, SUM(quantityOrdered) AS total_quantity
      -> FROM orderdetails
      -> GROUP BY productCode;
+-----+-----+
| productCode | total_quantity |
+-----+-----+
| P101        |             13 |
| P102        |              7 |
| P201        |              5 |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- 24. Total revenue per product
mysql> SELECT productCode,
      -> SUM(quantityOrdered * priceEach) AS revenue
      -> FROM orderdetails
      -> GROUP BY productCode;
+-----+-----+
| productCode | revenue |
+-----+-----+

```

```

+-----+-----+
| P101      | 390.00 |
| P102      | 196.00 |
| P201      | 75.00  |
+-----+-----+
3 rows in set (0.00 sec)

```

```

mysql> -- 25. Number of customers handled by each employee
mysql> SELECT salesRepEmployeeNumber, COUNT(*) AS total_customers
      -> FROM customers
      -> GROUP BY salesRepEmployeeNumber;

```

```

+-----+-----+
| salesRepEmployeeNumber | total_customers |
+-----+-----+
| 1002                   | 2              |
| 1003                   | 1              |
+-----+-----+
2 rows in set (0.00 sec)

```

```

mysql> -- 26. Customers with more than 5 orders
mysql> SELECT customerNumber, COUNT(orderNumber) AS total_orders
      -> FROM orders
      -> GROUP BY customerNumber
      -> HAVING COUNT(orderNumber) > 5;
Empty set (0.00 sec)

```

```

mysql> -- 27. Products with total quantity ordered > 100
mysql> SELECT productCode, SUM(quantityOrdered) AS total_quantity
      -> FROM orderdetails
      -> GROUP BY productCode
      -> HAVING SUM(quantityOrdered) > 100;
Empty set (0.00 sec)

```

```

mysql> -- 28. Employees managing more than 3 customers
mysql> SELECT salesRepEmployeeNumber, COUNT(*) AS total_customers
      -> FROM customers
      -> GROUP BY salesRepEmployeeNumber
      -> HAVING COUNT(*) > 3;
Empty set (0.00 sec)

```

```

mysql> -- 29. Countries with more than 10 customers
mysql> SELECT country, COUNT(*) AS total_customers
      -> FROM customers
      -> GROUP BY country
      -> HAVING COUNT(*) > 10;
Empty set (0.00 sec)

```

```

mysql> -- 30. Product lines having more than 5 products
mysql> SELECT productLine, COUNT(*) AS total_products
      -> FROM products
      -> GROUP BY productLine
      -> HAVING COUNT(*) > 5;
Empty set (0.00 sec)

```

```

mysql> -- 31. Top 5 customers by total payment
mysql> SELECT customerNumber, SUM(amount) AS total_payment
      -> FROM payments
      -> GROUP BY customerNumber
      -> ORDER BY total_payment DESC
      -> LIMIT 5;

```

```

+-----+-----+
| customerNumber | total_payment |
+-----+-----+
| 2002           | 500.00       |

```

Year	Revenue
2001	300.00
2003	150.00

3 rows in set (0.00 sec)

```
mysql> -- 32. Products ordered by total sales (revenue)
mysql> SELECT productCode,
  ->      SUM(quantityOrdered * priceEach) AS total_sales
  -> FROM orderdetails
  -> GROUP BY productCode
  -> ORDER BY total_sales DESC;
```

productCode	total_sales
P101	390.00
P102	196.00
P201	75.00

3 rows in set (0.00 sec)

```
mysql> -- 33. Employees sorted by number of customers handled
mysql> SELECT salesRepEmployeeNumber, COUNT(*) AS total_customers
  -> FROM customers
  -> GROUP BY salesRepEmployeeNumber
  -> ORDER BY total_customers DESC;
```

salesRepEmployeeNumber	total_customers
1002	2
1003	1

2 rows in set (0.00 sec)

```
mysql> -- 34. Countries sorted by total customers
mysql> SELECT country, COUNT(*) AS total_customers
  -> FROM customers
  -> GROUP BY country
  -> ORDER BY total_customers DESC;
```

country	total_customers
USA	1
UK	1
India	1

3 rows in set (0.00 sec)

```
mysql> -- 35. Top 10 most ordered products
mysql> SELECT productCode, SUM(quantityOrdered) AS total_orders
  -> FROM orderdetails
  -> GROUP BY productCode
  -> ORDER BY total_orders DESC
  -> LIMIT 10;
```

productCode	total_orders
P101	13
P102	7
P201	5

3 rows in set (0.00 sec)

```
mysql> -- 36. Customer name, order number, and product name
mysql> SELECT c.customerName, o.orderNumber, p.productName
```

```

-> FROM customers c
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> JOIN products p
->   ON od.productCode = p.productCode;

```

customerName	orderNumber	productName
ABC Corp	3001	Toyota Camry
ABC Corp	3001	Yamaha Bike
XYZ Ltd	3002	Honda Civic
Global Inc	3003	Toyota Camry

4 rows in set (0.00 sec)

mysql> -- 37. Employee name, customer name, and total payments

```

mysql> SELECT e.firstName, e.lastName,
->         c.customerName,
->         SUM(p.amount) AS total_payment
-> FROM employees e
-> JOIN customers c
->   ON e.employeeNumber = c.salesRepEmployeeNumber
-> JOIN payments p
->   ON c.customerNumber = p.customerNumber
-> GROUP BY e.employeeNumber, c.customerNumber;

```

firstName	lastName	customerName	total_payment
Jane	Doe	ABC Corp	300.00
Jane	Doe	Global Inc	150.00
Mike	Brown	XYZ Ltd	500.00

3 rows in set (0.00 sec)

mysql> -- 38. Order details with product line and quantity ordered

```

mysql> SELECT od.orderNumber,
->         p.productName,
->         p.productLine,
->         od.quantityOrdered
-> FROM orderdetails od
-> JOIN products p
->   ON od.productCode = p.productCode;

```

orderNumber	productName	productLine	quantityOrdered
3001	Toyota Camry	Cars	10
3003	Toyota Camry	Cars	3
3002	Honda Civic	Cars	7
3001	Yamaha Bike	Bikes	5

4 rows in set (0.01 sec)

mysql> -- 39. Office city with employee count

```

mysql> SELECT o.city, COUNT(e.employeeNumber) AS total_employees
-> FROM offices o
-> LEFT JOIN employees e
->   ON o.officeCode = e.officeCode
-> GROUP BY o.city;

```

city	total_employees
New York	2

```
| London | 1 |
+-----+-----+
2 rows in set (0.01 sec)
```

```
mysql> -- 40. Product vendor with total quantity sold
mysql> SELECT p.productVendor,
-> SUM(od.quantityOrdered) AS total_quantity
-> FROM products p
-> JOIN orderdetails od
-> ON p.productCode = od.productCode
-> GROUP BY p.productVendor;
```

```
+-----+-----+
| productVendor | total_quantity |
+-----+-----+
| Toyota        | 13             |
| Honda         | 7              |
| Yamaha        | 5              |
+-----+-----+
3 rows in set (0.00 sec)
```

```
mysql> -- 41. Average order value per customer
mysql> SELECT o.customerNumber,
-> AVG(order_total) AS avg_order_value
-> FROM (
-> SELECT o.orderNumber, o.customerNumber,
-> SUM(od.quantityOrdered * od.priceEach) AS order_total
-> FROM orders o
-> JOIN orderdetails od
-> ON o.orderNumber = od.orderNumber
-> GROUP BY o.orderNumber, o.customerNumber
-> ) AS t
-> GROUP BY customerNumber;
```

ERROR 1054 (42S22): Unknown column 'o.customerNumber' in 'field list'

```
mysql> -- 42. Total revenue generated per country
mysql> SELECT c.country,
-> SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM customers c
-> JOIN orders o
-> ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
-> ON o.orderNumber = od.orderNumber
-> GROUP BY c.country;
```

```
+-----+-----+
| country | revenue |
+-----+-----+
| USA     | 375.00 |
| UK      | 196.00 |
| India   | 90.00  |
+-----+-----+
```

3 rows in set (0.00 sec)

```
mysql> -- 43. Customers whose total payments exceed 50,000
mysql> SELECT customerNumber,
-> SUM(amount) AS total_payment
-> FROM payments
-> GROUP BY customerNumber
-> HAVING SUM(amount) > 50000;
Empty set (0.00 sec)
```

```
mysql> -- 44. Products where average order quantity > 50
mysql> SELECT productCode,
-> AVG(quantityOrdered) AS avg_qty
-> FROM orderdetails
-> GROUP BY productCode
```

```
-> HAVING AVG(quantityOrdered) > 50;
Empty set (0.00 sec)
```

```
mysql> -- 45. Top 3 product lines by total revenue
```

```
mysql> SELECT p.productLine,
->         SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM products p
-> JOIN orderdetails od
->   ON p.productCode = od.productCode
-> GROUP BY p.productLine
-> ORDER BY revenue DESC
-> LIMIT 3;
```

```
+-----+-----+
| productLine | revenue |
+-----+-----+
| Cars        | 586.00  |
| Bikes       | 75.00   |
+-----+-----+
2 rows in set (0.00 sec)
```

```
mysql> -- 46. Customers with credit limit > average credit limit
```

```
mysql> SELECT *
-> FROM customers
-> WHERE creditLimit > (
->     SELECT AVG(creditLimit) FROM customers
-> );
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
| customerNumber | customerName | contactLastName | contactFirstName | phone
| addressLine1 | addressLine2 | city | state | postalCode | country |
salesRepEmployeeNumber | creditLimit |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
| 2002 | XYZ Ltd | Kim | Pat |
222222222 | Street 2 | NULL | London | NULL | NW1 | UK
| 1003 | 70000.00 |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> -- 47. Orders where total amount > 10,000
```

```
mysql> SELECT o.orderNumber,
->         SUM(od.quantityOrdered * od.priceEach) AS total_amount
-> FROM orders o
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY o.orderNumber
-> HAVING total_amount > 10000;
Empty set (0.00 sec)
```

```
mysql> -- 48. Employees working in offices with more than 5 employees
```

```
mysql> SELECT *
-> FROM employees
-> WHERE officeCode IN (
->     SELECT officeCode
->     FROM employees
->     GROUP BY officeCode
->     HAVING COUNT(*) > 5
-> );
Empty set (0.00 sec)
```

mysql> -- 49. Products with stock less than average stock

```
mysql> SELECT *
-> FROM products
-> WHERE quantityInStock < (
->     SELECT AVG(quantityInStock) FROM products
-> );
```

```
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
| productCode | productName | productLine | productScale | productVendor |
productDescription | quantityInStock | buyPrice | MSRP |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
| P101        | Toyota Camry | Cars        | 1:18         | Toyota        |
Sedan model    |              | 100         | 20.00 | 30.00 |
+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

mysql> -- 50. Customers who made more than 3 payments

```
mysql> SELECT customerNumber,
->     COUNT(*) AS total_payments
-> FROM payments
-> GROUP BY customerNumber
-> HAVING COUNT(*) > 3;
```

Empty set (0.00 sec)

mysql> -- 51. Total revenue by each customer (with name, sorted highest)

```
mysql> SELECT c.customerName,
->     SUM(od.quantityOrdered * od.priceEach) AS total_revenue
-> FROM customers c
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY c.customerNumber, c.customerName
-> ORDER BY total_revenue DESC;
```

```
+-----+-----+
| customerName | total_revenue |
+-----+-----+
| ABC Corp     | 375.00        |
| XYZ Ltd      | 196.00        |
| Global Inc   | 90.00         |
+-----+-----+
```

3 rows in set (0.00 sec)

```
mysql> -- SELECT c.customerName,
mysql>     SUM(od.quantityOrdered * od.priceEach) AS total_revenue
-> FROM customers c
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY c.customerNumber, c.customerName
-> ORDER BY total_revenue DESC;
```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'SUM(od.quantityOrdered * od.priceEach) AS total_revenue FROM customers c JOIN or' at line 1

mysql> \c

mysql> -- 52. Customers who placed orders but made NO payments

```
mysql> SELECT DISTINCT c.customerName
-> FROM customers c
-> JOIN orders o
```

```

-> ON c.customerNumber = o.customerNumber
-> LEFT JOIN payments p
-> ON c.customerNumber = p.customerNumber
-> WHERE p.customerNumber IS NULL;
Empty set (0.00 sec)

```

mysql> -- 53. Total products ordered + total amount spent per customer

```

mysql> SELECT c.customerName,
-> SUM(od.quantityOrdered) AS total_products,
-> SUM(od.quantityOrdered * od.priceEach) AS total_spent
-> FROM customers c
-> JOIN orders o
-> ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
-> ON o.orderNumber = od.orderNumber
-> GROUP BY c.customerNumber, c.customerName;

```

```

+-----+-----+-----+
| customerName | total_products | total_spent |
+-----+-----+-----+
| ABC Corp     | 15             | 375.00      |
| XYZ Ltd      | 7              | 196.00      |
| Global Inc   | 3              | 90.00       |
+-----+-----+-----+

```

3 rows in set (0.00 sec)

mysql> -- v

mysql> -- 54. Product-wise revenue + total quantity sold (sorted)

```

mysql> SELECT p.productName,
-> SUM(od.quantityOrdered) AS total_quantity,
-> SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM products p
-> JOIN orderdetails od
-> ON p.productCode = od.productCode
-> GROUP BY p.productCode, p.productName
-> ORDER BY revenue DESC;

```

```

+-----+-----+-----+
| productName | total_quantity | revenue |
+-----+-----+-----+
| Toyota Camry | 13             | 390.00  |
| Honda Civic  | 7              | 196.00  |
| Yamaha Bike  | 5              | 75.00   |
+-----+-----+-----+

```

3 rows in set (0.00 sec)

mysql> -- 55. Employees with total sales generated from their customers

```

mysql> SELECT e.firstName, e.lastName,
-> SUM(od.quantityOrdered * od.priceEach) AS total_sales
-> FROM employees e
-> JOIN customers c
-> ON e.employeeNumber = c.salesRepEmployeeNumber
-> JOIN orders o
-> ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
-> ON o.orderNumber = od.orderNumber
-> GROUP BY e.employeeNumber, e.firstName, e.lastName
-> ORDER BY total_sales DESC;

```

```

+-----+-----+-----+
| firstName | lastName | total_sales |
+-----+-----+-----+
| Jane      | Doe      | 465.00      |
| Mike      | Brown    | 196.00      |
+-----+-----+-----+

```

2 rows in set (0.01 sec)

mysql> -- 56. Customers whose total purchase > average purchase of all customers

```
mysql> SELECT customerNumber,
->      SUM(od.quantityOrdered * od.priceEach) AS total_purchase
-> FROM orders o
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY customerNumber
-> HAVING total_purchase > (
->     SELECT AVG(total_per_customer)
->     FROM (
->       SELECT customerNumber,
->             SUM(od.quantityOrdered * od.priceEach) AS
total_per_customer
->       FROM orders o
->       JOIN orderdetails od
->         ON o.orderNumber = od.orderNumber
->       GROUP BY customerNumber
->     ) AS t
-> );
```

```
+-----+-----+
| customerNumber | total_purchase |
+-----+-----+
|          2001 |          375.00 |
+-----+-----+
1 row in set (0.00 sec)
```

mysql> -- 57. Products ordered in more than 2 different orders

```
mysql> SELECT productCode,
->      COUNT(DISTINCT orderNumber) AS order_count
-> FROM orderdetails
-> GROUP BY productCode
-> HAVING COUNT(DISTINCT orderNumber) > 2;
Empty set (0.00 sec)
```

mysql> -- 58. Customers who ordered more than 3 different products

```
mysql> SELECT o.customerNumber,
->      COUNT(DISTINCT od.productCode) AS product_count
-> FROM orders o
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY o.customerNumber
-> HAVING COUNT(DISTINCT od.productCode) > 3;
Empty set (0.00 sec)
```

mysql> -- 59. Product lines where total revenue exceeds 500

```
mysql> SELECT p.productLine,
->      SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM products p
-> JOIN orderdetails od
->   ON p.productCode = od.productCode
-> GROUP BY p.productLine
-> HAVING revenue > 500;
```

```
+-----+-----+
| productLine | revenue |
+-----+-----+
| Cars        |    586.00 |
+-----+-----+
1 row in set (0.00 sec)
```

```
mysql> -- SELECT p.productLine,
mysql>      SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM products p
-> JOIN orderdetails od
```

```

-> ON p.productCode = od.productCode
-> GROUP BY p.productLine
-> HAVING revenue > 500;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near 'SUM(od.quantityOrdered * od.priceEach) AS revenue
FROM products p
JOIN orderdeta' at line 1

```

```
mysql> -- 60. Countries where avg credit limit > 60,000
```

```
mysql> SELECT country,
->         AVG(creditLimit) AS avg_credit
-> FROM customers
-> GROUP BY country
-> HAVING AVG(creditLimit) > 60000;
```

```

+-----+-----+
| country | avg_credit |
+-----+-----+
| UK      | 70000.000000 |
+-----+-----+

```

```
1 row in set (0.01 sec)
```

```
mysql> -- 61. Customer name, order number, product name, quantity ordered
```

```
mysql> SELECT c.customerName,
->         o.orderNumber,
->         p.productName,
->         od.quantityOrdered
-> FROM customers c
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> JOIN products p
->   ON od.productCode = p.productCode;
```

```

+-----+-----+-----+-----+
| customerName | orderNumber | productName | quantityOrdered |
+-----+-----+-----+-----+
| ABC Corp    | 3001 | Toyota Camry | 10 |
| ABC Corp    | 3001 | Yamaha Bike  | 5  |
| XYZ Ltd     | 3002 | Honda Civic  | 7  |
| Global Inc  | 3003 | Toyota Camry | 3  |
+-----+-----+-----+-----+

```

```
4 rows in set (0.00 sec)
```

```
mysql> -- 62. Employee name, office city, number of customers handled
```

```
mysql> SELECT e.firstName, e.lastName,
->         o.city,
->         COUNT(c.customerNumber) AS total_customers
-> FROM employees e
-> JOIN offices o
->   ON e.officeCode = o.officeCode
-> LEFT JOIN customers c
->   ON e.employeeNumber = c.salesRepEmployeeNumber
-> GROUP BY e.employeeNumber, e.firstName, e.lastName, o.city;
```

```

+-----+-----+-----+-----+
| firstName | lastName | city      | total_customers |
+-----+-----+-----+-----+
| John     | Smith   | New York | 0 |
| Jane     | Doe     | New York | 2 |
| Mike     | Brown   | London   | 1 |
+-----+-----+-----+-----+

```

```
3 rows in set (0.01 sec)
```

```
mysql> -- 63. Product line, product name, total revenue generated
```

```
mysql> SELECT p.productLine,
```

```

->      p.productName,
->      SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM products p
-> JOIN orderdetails od
->   ON p.productCode = od.productCode
-> GROUP BY p.productLine, p.productName;
+-----+-----+-----+
| productLine | productName | revenue |
+-----+-----+-----+
| Cars        | Toyota Camry | 390.00 |
| Cars        | Honda Civic  | 196.00 |
| Bikes       | Yamaha Bike  | 75.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> -- 64. Order number, customer name, total order value
mysql> SELECT o.orderNumber,
->      c.customerName,
->      SUM(od.quantityOrdered * od.priceEach) AS total_value
-> FROM orders o
-> JOIN customers c
->   ON o.customerNumber = c.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY o.orderNumber, c.customerName;
+-----+-----+-----+
| orderNumber | customerName | total_value |
+-----+-----+-----+
|          3001 | ABC Corp    | 375.00 |
|          3002 | XYZ Ltd     | 196.00 |
|          3003 | Global Inc  | 90.00 |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> -- 65. Employees whose customers placed more than 5 orders
mysql> SELECT e.firstName, e.lastName,
->      COUNT(o.orderNumber) AS total_orders
-> FROM employees e
-> JOIN customers c
->   ON e.employeeNumber = c.salesRepEmployeeNumber
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> GROUP BY e.employeeNumber, e.firstName, e.lastName
-> HAVING COUNT(o.orderNumber) > 5;
Empty set (0.00 sec)

mysql> -- 66. Customers whose total payments < total order value
mysql> SELECT c.customerNumber, c.customerName,
->      COALESCE(SUM(p.amount),0) AS total_payments,
->      SUM(od.quantityOrdered * od.priceEach) AS total_orders
-> FROM customers c
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> LEFT JOIN payments p
->   ON c.customerNumber = p.customerNumber
-> GROUP BY c.customerNumber, c.customerName
-> HAVING total_payments < total_orders;
Empty set (0.00 sec)

mysql> -- 67. Products where total quantity sold < average across all
products
mysql> SELECT productCode,

```

```

->         SUM(quantityOrdered) AS total_qty
-> FROM orderdetails
-> GROUP BY productCode
-> HAVING total_qty < (
->     SELECT AVG(total_qty)
->     FROM (
->         SELECT SUM(quantityOrdered) AS total_qty
->         FROM orderdetails
->         GROUP BY productCode
->     ) t
-> );

```

```

+-----+-----+
| productCode | total_qty |
+-----+-----+
| P102        |          7 |
| P201        |          5 |
+-----+-----+

```

2 rows in set (0.00 sec)

mysql> -- 68. Customers with NO orders but HAVE payments

```

mysql> SELECT c.customerName
-> FROM customers c
-> LEFT JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN payments p
->   ON c.customerNumber = p.customerNumber
-> WHERE o.customerNumber IS NULL;

```

Empty set (0.00 sec)

mysql> -- 69. Employees with NO customers

```

mysql> SELECT e.firstName, e.lastName
-> FROM employees e
-> LEFT JOIN customers c
->   ON e.employeeNumber = c.salesRepEmployeeNumber
-> WHERE c.customerNumber IS NULL;

```

```

+-----+-----+
| firstName | lastName |
+-----+-----+
| John      | Smith    |
+-----+-----+

```

1 row in set (0.00 sec)

mysql> -- 70. Orders with more than 1 product (multi-line)

```

mysql> SELECT orderNumber,
->     COUNT(productCode) AS product_count
-> FROM orderdetails
-> GROUP BY orderNumber
-> HAVING COUNT(productCode) > 1;

```

```

+-----+-----+
| orderNumber | product_count |
+-----+-----+
|          3001 |          2 |
+-----+-----+

```

1 row in set (0.00 sec)

mysql> 71. Top 3 customers by total order value

```

-> SELECT c.customerName,
->     SUM(od.quantityOrdered * od.priceEach) AS total_value
-> FROM customers c
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY c.customerNumber, c.customerName

```

```

-> ORDER BY total_value DESC
-> LIMIT 3;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual
that corresponds to your MySQL server version for the right syntax to use
near '71. Top 3 customers by total order value
SELECT c.customerName,
      SUM(od.qu' at line 1
mysql> -- 72. Top 5 products by total revenue
mysql> SELECT p.productName,
->      SUM(od.quantityOrdered * od.priceEach) AS revenue
-> FROM products p
-> JOIN orderdetails od
->   ON p.productCode = od.productCode
-> GROUP BY p.productCode, p.productName
-> ORDER BY revenue DESC
-> LIMIT 5;
+-----+-----+
| productName | revenue |
+-----+-----+
| Toyota Camry | 390.00 |
| Honda Civic  | 196.00 |
| Yamaha Bike  | 75.00  |
+-----+-----+
3 rows in set (0.00 sec)

mysql> -- 73. Top 2 employees by sales generated
mysql> SELECT e.firstName, e.lastName,
->      SUM(od.quantityOrdered * od.priceEach) AS total_sales
-> FROM employees e
-> JOIN customers c
->   ON e.employeeNumber = c.salesRepEmployeeNumber
-> JOIN orders o
->   ON c.customerNumber = o.customerNumber
-> JOIN orderdetails od
->   ON o.orderNumber = od.orderNumber
-> GROUP BY e.employeeNumber, e.firstName, e.lastName
-> ORDER BY total_sales DESC
-> LIMIT 2;
+-----+-----+-----+
| firstName | lastName | total_sales |
+-----+-----+-----+
| Jane      | Doe      | 465.00      |
| Mike      | Brown    | 196.00      |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> -- 74. Top 3 countries by number of customers
mysql> SELECT country, COUNT(*) AS total_customers
-> FROM customers
-> GROUP BY country
-> ORDER BY total_customers DESC
-> LIMIT 3;
+-----+-----+
| country | total_customers |
+-----+-----+
| USA     | 1               |
| UK      | 1               |
| India   | 1               |
+-----+-----+
3 rows in set (0.00 sec)

mysql> -- 75. Top 5 most expensive products (MSRP)
mysql> SELECT productName, MSRP
-> FROM products

```

```

-> ORDER BY MSRP DESC
-> LIMIT 5;

```

```

+-----+-----+
| productName | MSRP |
+-----+-----+
| Toyota Camry | 30.00 |
| Honda Civic  | 28.00 |
| Yamaha Bike  | 15.00 |
+-----+-----+
3 rows in set (0.00 sec)

```

```
mysql> -- 76. Avg order value per customer > overall avg
```

```
mysql> SELECT customerNumber, AVG(order_total) AS avg_value
```

```

-> FROM (
->     SELECT o.customerNumber,
->           o.orderNumber,
->           SUM(od.quantityOrdered * od.priceEach) AS order_total
->     FROM orders o
->     JOIN orderdetails od
->       ON o.orderNumber = od.orderNumber
->     GROUP BY o.orderNumber, o.customerNumber
-> ) t
-> GROUP BY customerNumber
-> HAVING avg_value > (
->     SELECT AVG(order_total)
->     FROM (
->         SELECT SUM(od.quantityOrdered * od.priceEach) AS order_total
->         FROM orders o
->         JOIN orderdetails od
->           ON o.orderNumber = od.orderNumber
->         GROUP BY o.orderNumber
->     ) t2
-> );

```

```

+-----+-----+
| customerNumber | avg_value |
+-----+-----+
| 2001 | 375.000000 |
+-----+-----+
1 row in set (0.00 sec)

```

```
mysql> -- 77. Product-wise avg price sold vs MSRP difference
```

```

mysql> SELECT p.productName,
->       AVG(od.priceEach) AS avg_price_sold,
->       p.MSRP,
->       (p.MSRP - AVG(od.priceEach)) AS price_diff
-> FROM products p
-> JOIN orderdetails od
->   ON p.productCode = od.productCode
-> GROUP BY p.productCode, p.productName, p.MSRP;

```

```

+-----+-----+-----+-----+
| productName | avg_price_sold | MSRP | price_diff |
+-----+-----+-----+-----+
| Toyota Camry | 30.000000 | 30.00 | 0.000000 |
| Honda Civic  | 28.000000 | 28.00 | 0.000000 |
| Yamaha Bike  | 15.000000 | 15.00 | 0.000000 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

```
mysql> -- 78. % contribution of each product to total revenue
```

```

mysql> SELECT p.productName,
->       SUM(od.quantityOrdered * od.priceEach) AS revenue,
->       (SUM(od.quantityOrdered * od.priceEach) /
->       (SELECT SUM(quantityOrdered * priceEach) FROM orderdetails) *
100)

```

```

->          AS percentage
-> FROM products p
-> JOIN orderdetails od
->   ON p.productCode = od.productCode
-> GROUP BY p.productCode, p.productName;

```

```

+-----+-----+-----+
| productName | revenue | percentage |
+-----+-----+-----+
| Toyota Camry | 390.00 | 59.001513 |
| Honda Civic  | 196.00 | 29.652042 |
| Yamaha Bike  | 75.00  | 11.346445 |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

mysql> -- 79. Avg quantity ordered per order for each customer

```

mysql> SELECT customerNumber,
->          AVG(total_qty) AS avg_qty_per_order
-> FROM (
->   SELECT o.customerNumber,
->          o.orderNumber,
->          SUM(od.quantityOrdered) AS total_qty
->   FROM orders o
->   JOIN orderdetails od
->     ON o.orderNumber = od.orderNumber
->   GROUP BY o.orderNumber, o.customerNumber
-> ) t
-> GROUP BY customerNumber;

```

```

+-----+-----+-----+
| customerNumber | avg_qty_per_order |
+-----+-----+-----+
|          2001 |          15.0000 |
|          2002 |           7.0000 |
|          2003 |           3.0000 |
+-----+-----+-----+
3 rows in set (0.00 sec)

```

mysql> -- 80. Customers whose largest single order > 10,000

```

mysql> SELECT customerNumber,
->          MAX(order_total) AS max_order_value
-> FROM (
->   SELECT o.customerNumber,
->          o.orderNumber,
->          SUM(od.quantityOrdered * od.priceEach) AS order_total
->   FROM orders o
->   JOIN orderdetails od
->     ON o.orderNumber = od.orderNumber
->   GROUP BY o.orderNumber, o.customerNumber
-> ) t
-> GROUP BY customerNumber
-> HAVING MAX(order_total) > 10000;

```

Empty set (0.00 sec)

mysql>